

0.11. Tout l'alphabet, avec textS

textS passe à la ligne tout seul : plus de calcul à écrire.

1. Ce qu'il faut comprendre

`textS` écrit comme `texte`, mais **passse à la ligne** quand la ligne est pleine : après la colonne 19, la suite reprend en colonne 0 de la ligne d'en dessous.

Le calcul du 0.9 (`i % 20`, `i / 20`) est fait **à notre place**. `texte()`, lui, refuserait ce mot trop long.

Après la ligne 17, `textS` repart **en haut**, à la ligne 0.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1321 octets** de cartouche.

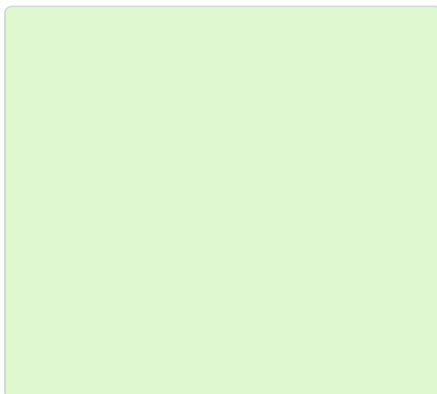
1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    textS(0, 0, "ABCDEFGHIJKLMNOPQRSTUVWXYZ"); // passe à la ligne après T

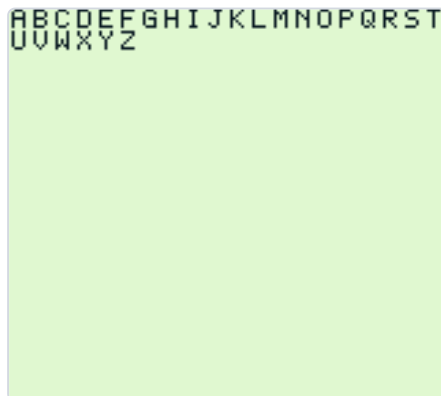
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0, U à Z en dessous, comme au 0.9.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 1 : U à Z

Ensuite : **0.11.1. Tout l'alphabet, avec textS — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++